COMP 018: Database Administration - Activity 5 Progress Report
Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: PUP-SPOT: Campus Lost and Found System
Target Project Group	: Group 5 (Imperial, Manansala, Napeñas, Lctz, Quody)
Target Project Repository	: https://github.com/grp05-dba/grp05_lostandfound
Primary Database	: grp05_lostandfounddb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	: grp05_lostandfounddb_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection
Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 8.6 megabytes across eight base tables and eight sequences within the public schema. Table storage analysis showed that lost reports and claim tables account for 48 kilobytes each, the items catalog accounts for 32 kilobytes, and potential matches, item images, and categories account for 24 kilobytes each. Supporting image tables for lost reports and claims occupy minimal storage space.

2.2 Connection, Session & Activity Monitoring
Database statistics recorded one active connection with 63 committed transactions and 7 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across sessions, confirming stable transaction processing. The rolled back transaction count reflected expected error conditions intentionally triggered during negative security verification testing. System catalog queries confirmed that active roles and memberships accurately reflect access control rules.

2.3 Table Statistics & Bloat Inspection
Pre-maintenance table statistics revealed dead tuple accumulation within the items catalog and item images table resulting from update and deletion operations executed during Activity 4 verification testing. Obsolete row versions across active relations confirmed the necessity of running routine vacuuming and statistics updating operations.

2.4 Log Review & Issue Identification
Review of database logs verified that unauthorized access attempts during Activity 4 security testing were recorded by the engine. Logged events included permission denials on update,

insertion, and deletion commands attempted under public user credentials, blocked database creation requests, and denied cross-database connection attempts. Server background processes completed cleanly without system crashes or storage warnings.

2.5 Routine Maintenance Operations

Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Initial testing confirmed that non-owner application runtime accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative credentials.

Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming on the items catalog scanned the relation, removing obsolete row versions and updating frozen transaction counts. Vacuum processing on the item images table removed dead tuples and truncated empty pages, restoring storage efficiency.

Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application read and write operations. Because relations reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples effectively without disrupting application availability.

2.6 Query Performance Analysis

Performance analysis was conducted on a public query filtering unclaimed items by category and sorting results by date found. The query executed in 0.089 milliseconds with complete buffer cache hit efficiency. The query planner selected sequential scans over index lookups because underlying data fits inside a single memory page, avoiding index page processing overhead.

Analysis of a staff review query joining potential matches, lost reports, and item catalog entries yielded an execution time of 0.141 milliseconds with zero physical disk reads. The query engine built in-memory hash tables across join relations, executing chained hash joins within dedicated memory structures without disk spills.

2.7 Prioritized Recommendations & Technical Justification

The current index configuration is appropriate for current operational requirements, as active queries execute under 0.15 milliseconds with complete buffer cache hits. Immediate index additions are unnecessary and would introduce write overhead during report filings. If lost item and claim volumes increase significantly in production, adding B-Tree indexes on foreign key columns will optimize join processing. Deploying a scheduled weekly background maintenance task will maintain page efficiency during low-traffic periods.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 5 - PUP-SPOT: Campus Lost and Found System
Database inspected	grp05_lostandfounddb (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), lostandfound_app (Staff Admin), lostandfound_public (Public User)
Health check result	Healthy (0 conflicts, 0 deadlocks, 8.6 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on items, lost_reports, item_images via postgres
Query analyzed	Public items search by category & status (Seq scan + Hash join, 0.089 ms) & Multi-table match review (Hash joins, 0.141 ms)
Recommendation property	No action required (Current queries execute in < 0.15 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	Yes (05/11/2026)
PDF progress document	Activity5_Progress_Report_Group5.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.